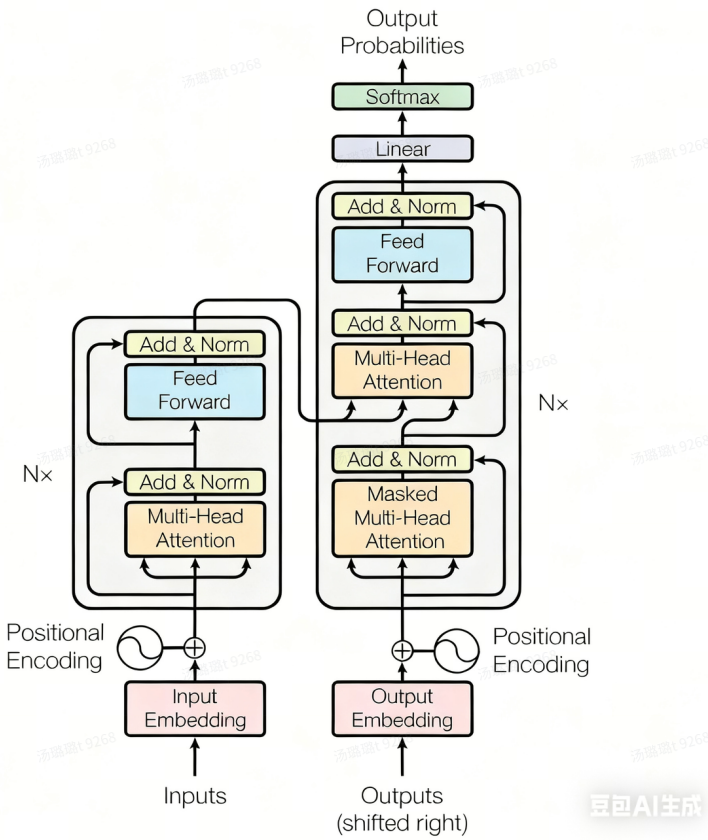
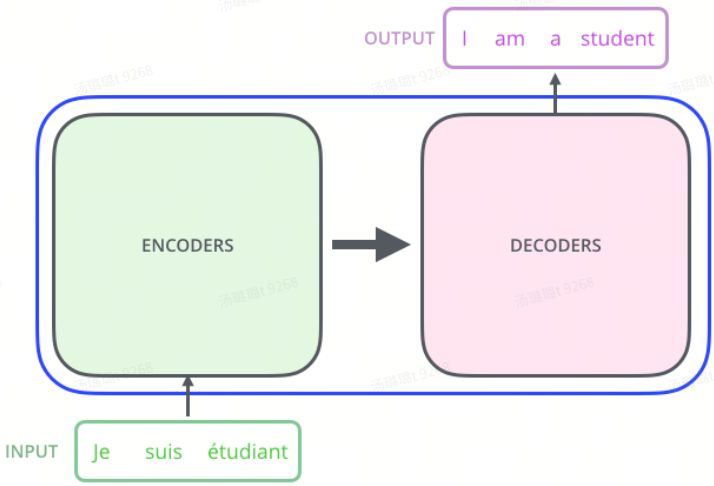
Transformer分享

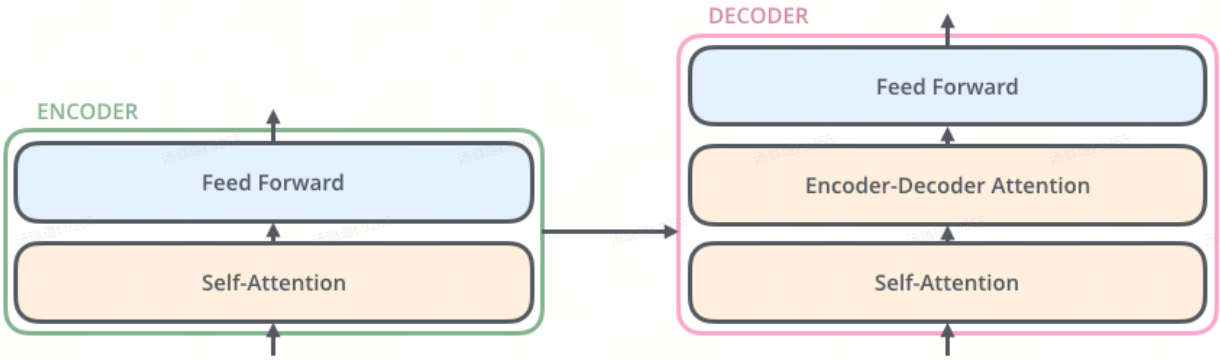


宏观理解Transformer核心结构

- 在机器翻译任务中，模型视为黑盒，接收一种语言的句子作为输入，然后将其翻译成其他语言输出。
- 该流程的核心 ——Transformer，可拆分为两部分：左侧为**编码部分（encoding component）**，右侧为**解码部分（decoding component）**。



- 其中，编码部分由**多层编码器（Encoder）** 组成；同理，解码部分由**多层解码器（Decoder）**组成。
- 每一个编码器的**结构完全相同**，但各自的**权重参数存在差异**。单个编码器内部包含两层核心结构，具体如下：
  - Self-Attention Layer（自注意力层）
  - Feed Forward Neural Network（前馈神经网络，缩写为 FFNN）
- 输入**编码器**的文本数据，首先进入 Self-Attention 层，该层处理单个词时，不仅会利用该词自身的信息，还会结合句子中其他所有词的信息；随后，Self-Attention 层的输出会传入前馈神经网络，完成进一步处理。
- **解码器**同样包含上述 Self-Attention 层与 FFNN 两层结构，但不同的是，这两层之间额外插入了一层 Encoder-Decoder Attention 层—— 该层能帮助解码器在生成目标语言内容时，精准聚焦于输入句子中与之相关的部分。

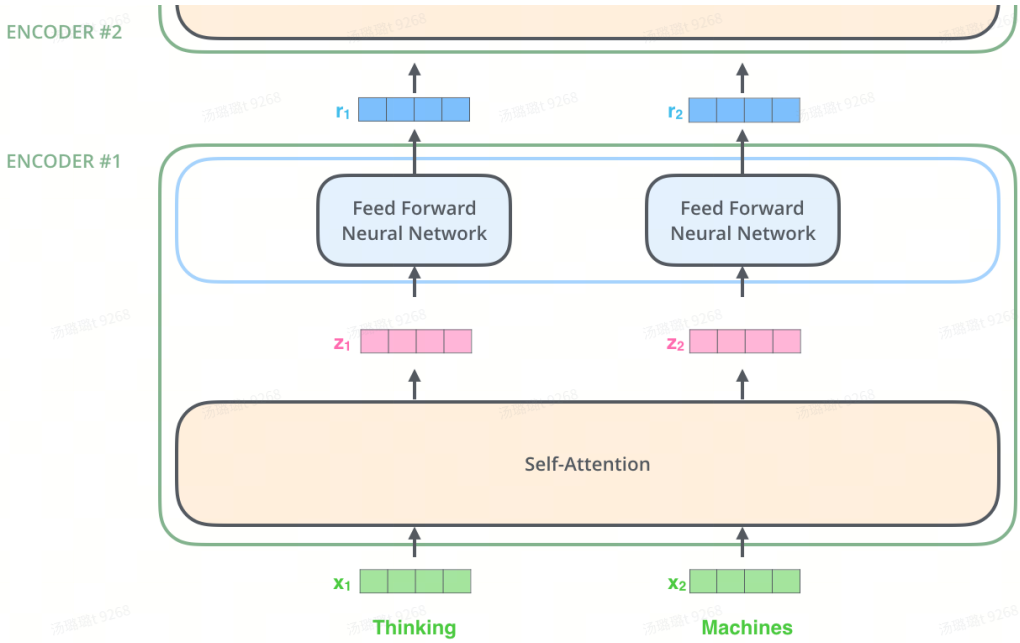


细节理解Transformer的输入到输出

Transformer的输入

i. 在 Transformer 模型中，输入处理流程与常规 NLP 任务一致，首先通过**词嵌入算法（Embedding Algorithm）**，将句子中的每个离散词汇，转换为词向量。那么整个输入的句子是一个向量列表，其中有 3 个词向量。在实际中，每个句子的长度不一样，我们会取一个适当的值，作为向量列表的长度。如果一个句子达不到这个长度，那么就填充全为 0 的词向量；如果句子超出这个长度，则做截断。句子长度是一个超参数，通常是训练集中的句子的最大长度，你可以尝试不同长度的效果。

ii. **编码器（Encoder）接收**的输入都是一个向量列表，输出也是大小同样的向量列表，然后接着输入下一个编码器。第一个编码器的输入是词向量，而后面的编码器的输入是上一个编码器的输出。

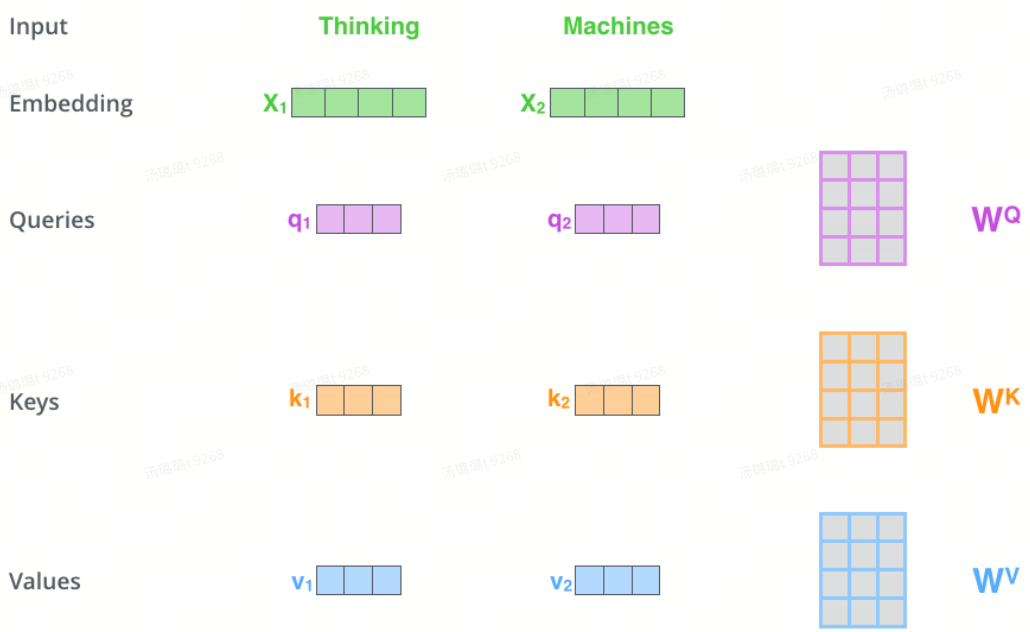


Self-Attention 的核心细节

（一）分步解析：以单个词汇为例

假设输入句子包含两个词汇（如 “Thinking” 和 “Machines”），对应的词向量分别为 x_1（Thinking）和 x_2（Machines），Self-Attention 的计算需经历 6 个关键步骤：

1. **生成 Query、Key、Value 向量**



   a. 对每个词向量，通过与 3 个**可学习的权重矩阵**（`W_Q`、`W_K`、`W_V`）相乘，生成 3 个新向量：

      - Query 向量（查询向量）：词向量与 `W_Q` 相乘得到，用于 “查询” 其他词汇的关联信息，如 $x_1 \times W_Q = q_1$（q_1 为 Thinking 的 Query 向量）；

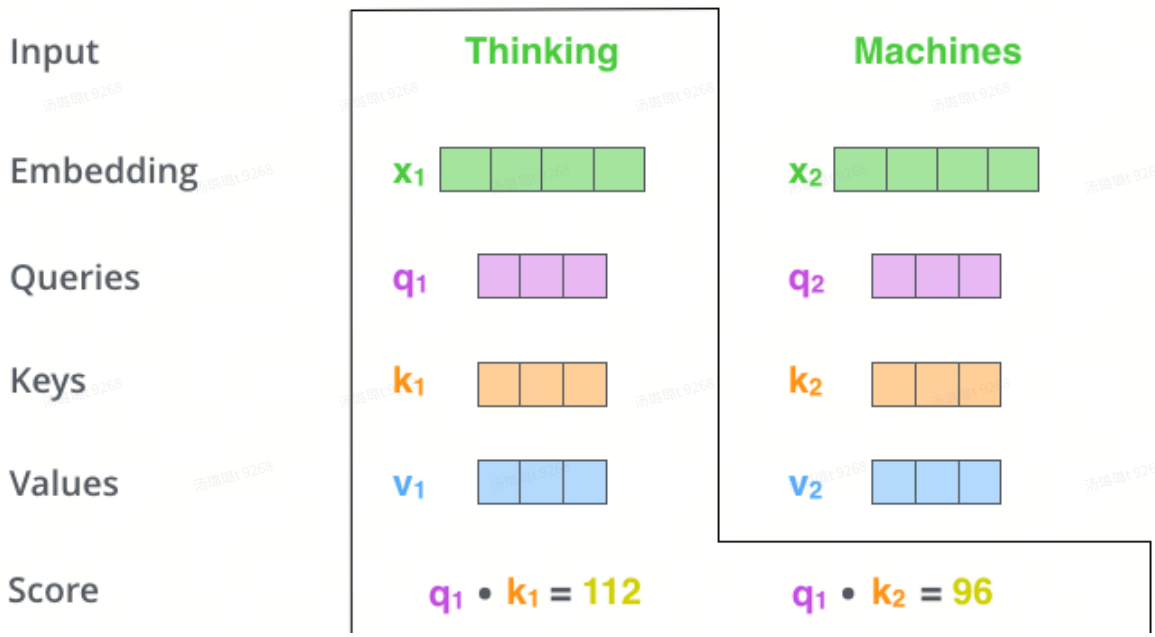
- Key 向量（键向量）：词向量与 `W_K` 相乘得到，用于 “匹配” Query 向量的查询需求，如 $x_1 \times W_K = k_1$ （k_1 为 Thinking 的 Key 向量）；
- Value 向量（值向量）：词向量与 `W_V` 相乘得到，是最终参与 “注意力权重分配” 的核心向量，如 $x_1 \times W_V = v_1$ （v_1 为 Thinking 的 Value 向量）。

2. **计算 Attention Score（注意力分数）**

> 📌 注意力分数的作用：**衡量当前词汇与句子中其他所有词汇的关联程度。**

a. 以计算 “Thinking（x_1）” 的注意力分数为例：

- 用 x_1 的 Query 向量（q_1），分别与句子中每个词汇的 Key 向量（k_1、k_2）做**点积运算**；
- 得到的结果即为注意力分数：第一个分数是 $q_1 \times k_1$（当前词与自身的关联度），第二个分数是 $q_1 \times k_2$ （当前词与 “Machines” 的关联度）。



3. **分数归一化（除以Key 向量维度的平方根（√d_k））**

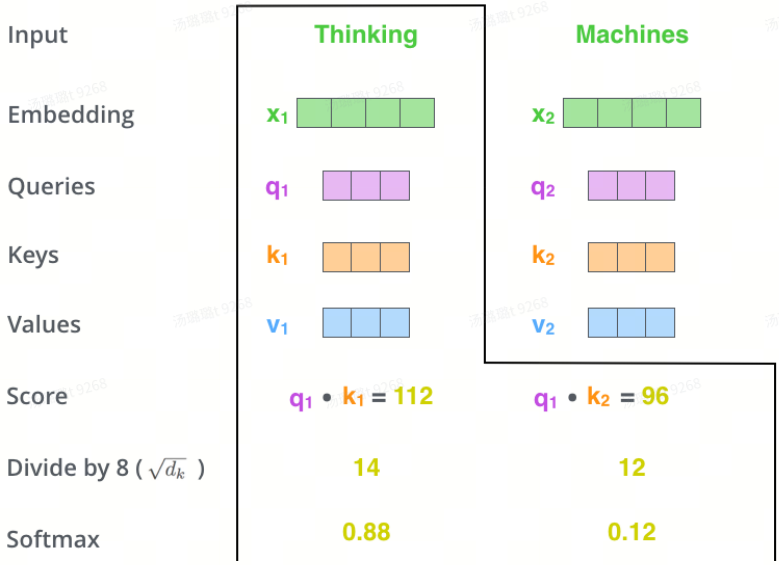
a. 将第 2 步得到的所有注意力分数，统一除以**Key 向量的维度平方根**（如 Key 向量长度为 64 时，除以√64=8）。（防止注意力分数过大，除以√d_k 能让分数方差更稳定）

4. **Softmax 层处理**

a. 将归一化后的分数输入**Softmax 函数**，完成两件事：

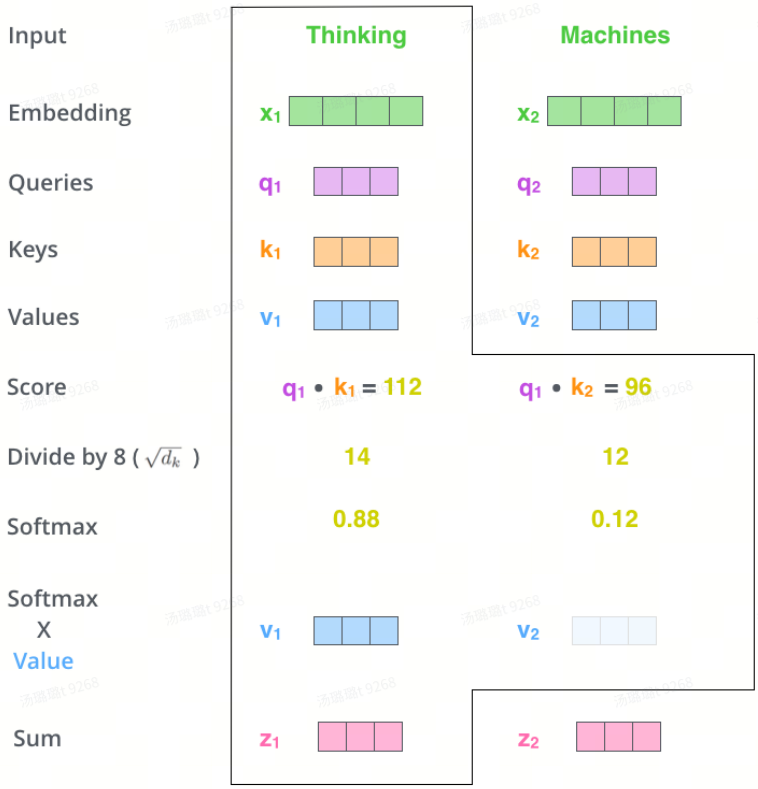
i. 把所有分数转换为 “正数”；

ii. 让所有分数的和为 1（实现概率化的权重分配）。



- 这些分数决定了在编码当前位置（这里的例子是第一个位置）的词时，对所有位置的词分别有多少的注意力。很明显，在上图的例子中，当前词汇的权重最高，但当其他词汇与当前词语义相关时（如 “猫” 和 “狗”），相关词汇的权重也会显著提升。

5. **“权重” 与 Value 向量相乘**
   a. 将 Softmax 输出的 “注意力权重”，分别与对应词汇的 Value 向量相乘。
   b. 做法背后的直觉理解就是：对于分数高的位置，相乘后的值就越大，我们把更多的注意力放到了它们身上；对于分数低的位置，相乘后的值就越小，这些位置的词可能是相关性不大的，这样我们就忽略了这些位置的词。
   c. 例如，若 “Thinking” 对自身的权重为 0.8，对 “Machines” 的权重为 0.2，则计算为：`0.8 × v₁ + 0.2 × v₂`。
6. **向量求和，得到 Self-Attention 输出**
   a. 将第 5 步中 “权重 ×Value 向量” 的所有结果相加，得到的向量即为**Self Attention 层在这个位置（对第一个单词）的输出。**
   b. 该输出向量会直接传入后续的 “前馈神经网络”，继续完成特征加工。



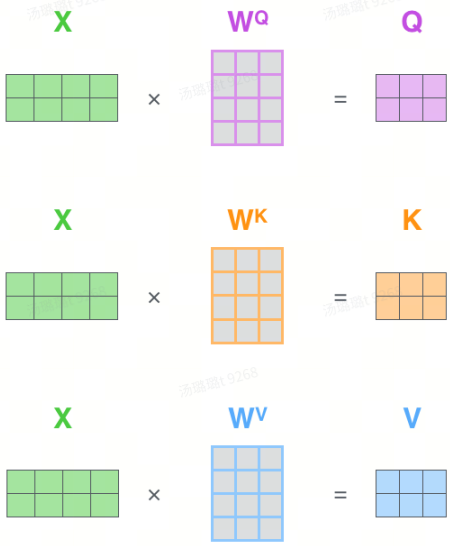
- 上面这张图，包含了 Self Attention 的全过程，最终得到的当前位置（这里的例子是第一个单词）的向量会输入到前馈神经网络。但这样每次只能计算一个位置的输出向量。

（二）矩阵计算：批量加速 Self-Attention

- 实际工程中为了提升计算效率，会用**矩阵运算**一次性处理句子中所有词汇的 Self-Attention，一次就得到所有位置的输出向量。

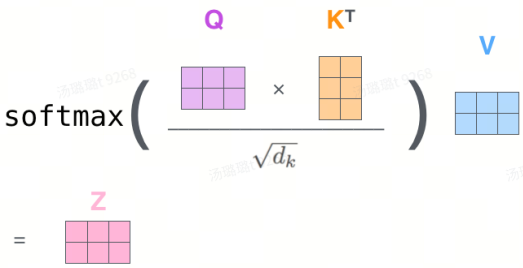
i. 第一步是计算 Query，Key，Value 的矩阵。

1. 将句子中所有词汇的词向量，按 “行” 堆叠成一个矩阵 `X` ，分别将 `X` 与权重矩阵 `W_Q` 、 `W_K` 、 `W_V` 相乘，得到得到 Q，K，V 矩阵。
2. 矩阵 X 中的每一行，表示句子中的每一个词的词向量。Q，K，V 矩阵中的每一行表示 Query 向量，Key 向量，Value 向量。



ii. 一次性计算所有词汇的 Self-Attention 输出

1. 由于我们使用了矩阵来计算，我们可以把上面的第 2 步到第 6 步压缩为一步，直接得到 Self Attention 的输出。



多头注意力机制

- 多头注意力机制是 Transformer 模型中对 Self Attention 层的关键优化，通过 “多组注意力头（attention head）并行计算” 的方式，从两个核心维度增强 Attention 层的能力，为后续特征处理提供更全面的语义信息。

(一) 核心能力：两大增强作用

1. 扩展 “多位置关注” 能力

   a. 在传统 Self Attention 中，单个输出向量（如上面的例子中第一个位置的输出 z_1）虽包含其他位置信息，但主要由自身位置信息主导，难以精准捕捉远距离依赖关系。

   b. **典型场景**：翻译句子 “The animal didn’t cross the street because it was too tired” 时，需明确 “it” 的指代对象（“the animal”）。多头注意力机制通过多组注意力头并行关注不同位置，可精准定位 “it” 与 “the animal” 的关联，解决远距离指代问题。

2. 构建 “多子表示空间”

   多头注意力机制通过多组独立的权重矩阵，将输入向量映射到多个不同的 “子表示空间”：

- Transformer 论文中默认使用 8 组注意力头，每组均对应独立的权重矩阵（WQ、WK、WV），且初始值随机；
- 经过训练后，每组注意力头会形成独特的 “子表示空间”，可捕捉输入数据中不同维度的特征（如语义、语法、情感等），让模型对输入的理解更立体。

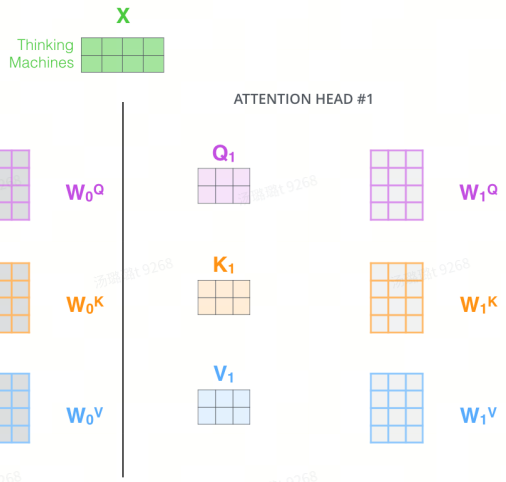
(二) 实现流程：三步完成多头注意力计算

接下来用 8 组注意力头 （attention heads）为例。具体计算步骤如下：

i. 步骤 1：生成多组 Q、K、V 矩阵

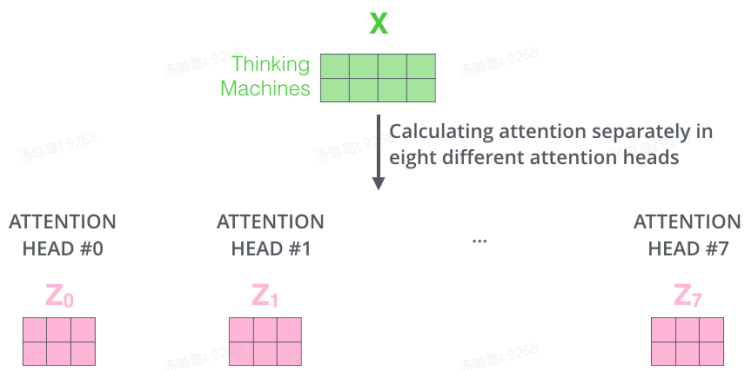
为 8 组注意力头分别维护独立的权重矩阵（$WQ_0/WK_0/WV_0$、$WQ_1/WK_1/WV_1$……$WQ_7/WK_7/WV_7$）：

将输入矩阵 X 分别与每组的 WQ、WK、WV 相乘，得到 8 组对应的 Q（查询矩阵）、K（键矩阵）、V（值矩阵），即（Q_0,K_0,V_0）、（Q_1,K_1,V_1）……（Q_7,K_7,V_7）。



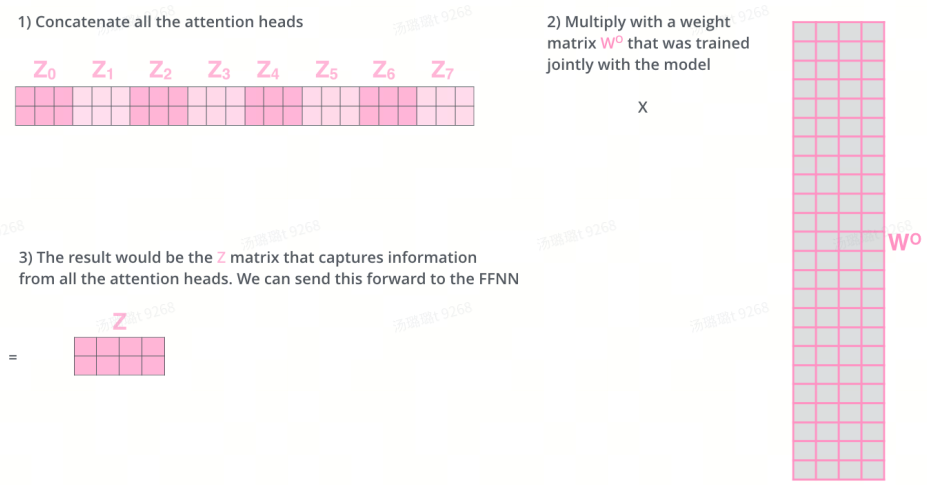
ii. 步骤 2：计算每组的输出矩阵 Z

对 8 组（Q,K,V）分别执行传统 Self Attention 的计算逻辑（通过 Q 与 K 的相似度计算注意力权重，再结合 V 加权求和），得到 8 个独立的输出矩阵：Z_0、Z_1、Z_2……Z_7。



iii. 步骤 3：拼接多组 Z 矩阵

1. 由于后续的前馈神经网络（FFNN）仅接收 “单个矩阵” （每行向量对应一个词的特征），需先将 8 个 Z 矩阵按维度拼接，形成一个整合了所有注意力头信息的 “拼接矩阵”。
2. 为让拼接后的矩阵符合 FFNN 的输入维度要求，将 “拼接矩阵” 与一个额外的权重矩阵 WO 相乘，得到最终的输出矩阵 Z。该矩阵 Z 融合了 8 组注意力头的所有信息，可直接输入 FFNN 层进行后续特征加工。

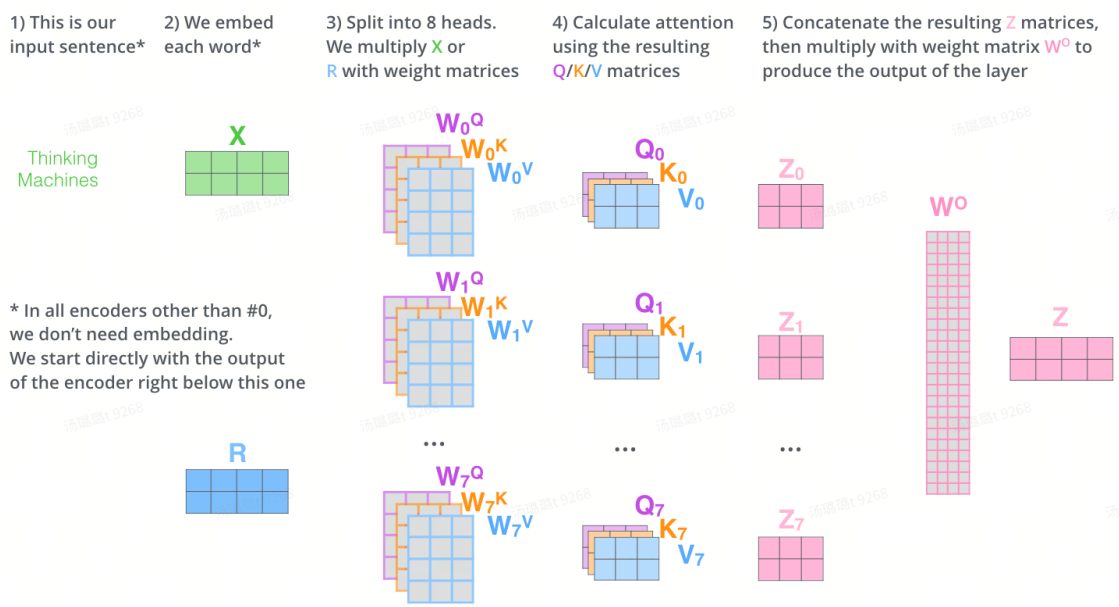


(三) 效果示例：以 “it” 的编码为例

翻译句子 “The animal didn’t cross the street because it was too tired” 时，当模型编码单词 “it” 时，8 组注意力头会呈现差异化关注：

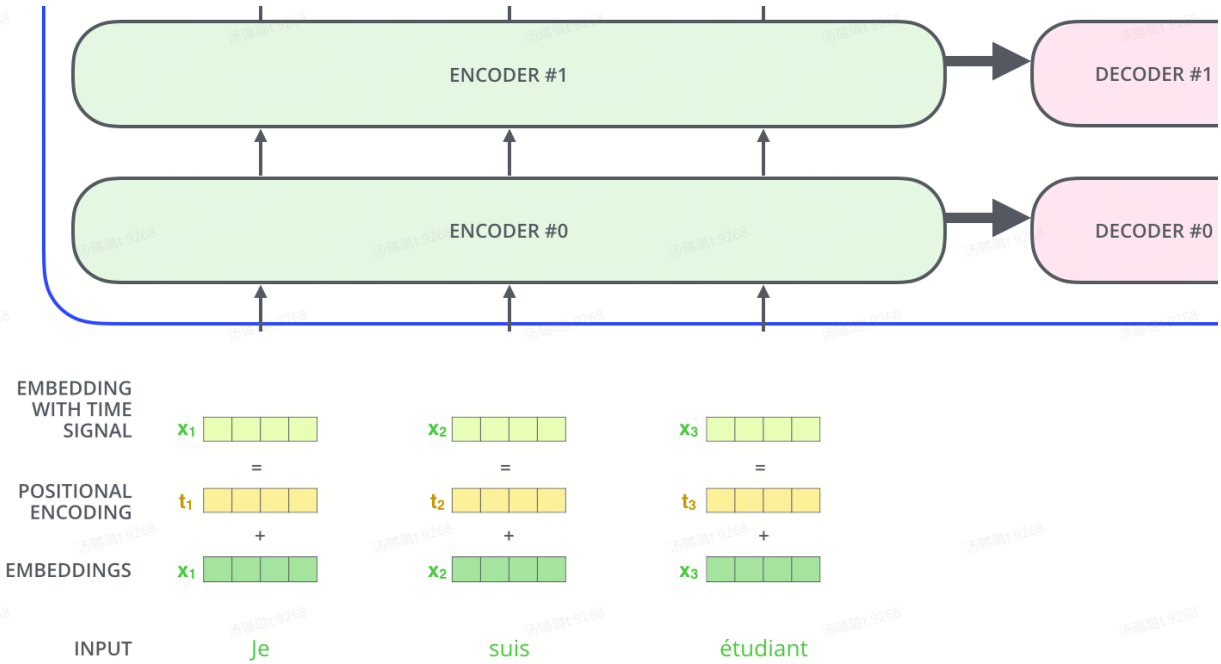
- 某一组注意力头主要关注 “the animal” （明确指代对象）；
- 另一组注意力头主要关注 “tired” （理解 “it” 的状态）；

最终通过步骤 3-4 的整合，“it” 在模型中的特征表示会融合 “animal” 的指代信息与 “tired” 的状态信息，实现更精准的语义编码。



位置编码

- 为什么需要位置编码?
  - 在 Transformer 模型的 Self Attention 与多头注意力机制中，存在一个关键缺陷：**模型无法感知输入序列中单词的顺序信息**。
  - 传统序列模型（如 RNN、LSTM）通过 “逐词迭代计算” 自然捕捉顺序，但 Transformer 的 Attention 层通过并行计算处理所有词向量，若不额外干预，会将 “the cat eats fish” 与 “fish eats the cat” 视为相同的输入（仅词向量集合一致，无顺序差异）。
  - 为解决这一问题，Transformer 引入 “位置编码” 机制，为每个词向量注入独有的顺序信息，确保模型能区分序列中单词的位置及彼此间的距离。



- 正弦余弦位置编码
  - 为了让模型感知序列顺序，Transformer论文引入了正弦余弦位置编码。公式如下：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

  - 其中 pos 是序列位置， i 是维度索引， d_{model} 是模型维度。这种编码方式通过不同频率的正弦余弦函数生成位置信息，使模型能够通过简单线性变换感知位置间的相对关系。
  - 优点：

a. 相对位置可计算性：根据三角函数的和角公式（$\sin(a+b)=\sin a\cos b+\cos a\sin b$ 和 $\cos(a+b)=\cos a\cos b-\sin a\sin b$），两个位置 `pos` 和 `pos+k` 的位置向量，可通过线性变换关联 —— 这意味着模型能直接学到 “相对位置关系”（如 “pos+3” 与 “pos” 的距离是 3），而可学习编码需重新学习不同长度序列的位置关联。

b. 长序列泛化性：训练时若仅见过短序列（如最长 512token），正弦余弦编码可直接通过公式生成 “更长序列（如 1024token）的位置向量”，无需额外训练；而可学习编码对未见过的长序列，位置信息会 “失效”（因为训练中未更新过超长位置的向量）。

前馈神经网络

- 作用：前馈神经网络是 Transformer 编码器 / 解码器中注意力层之后的固定模块，其核心作用是对注意力层输出的 “全局关联特征” 进行进一步的非线性加工，增强模型对局部特征的捕捉能力（弥补注意力层更侧重全局依赖的特点）。
- Transformer 论文中定义的 FFN 是一个两层全连接网络，且对序列中每个词的特征向量独立处理（即 “逐位置” 变换，无序列间依赖，可并行计算）。其公式与流程如下：

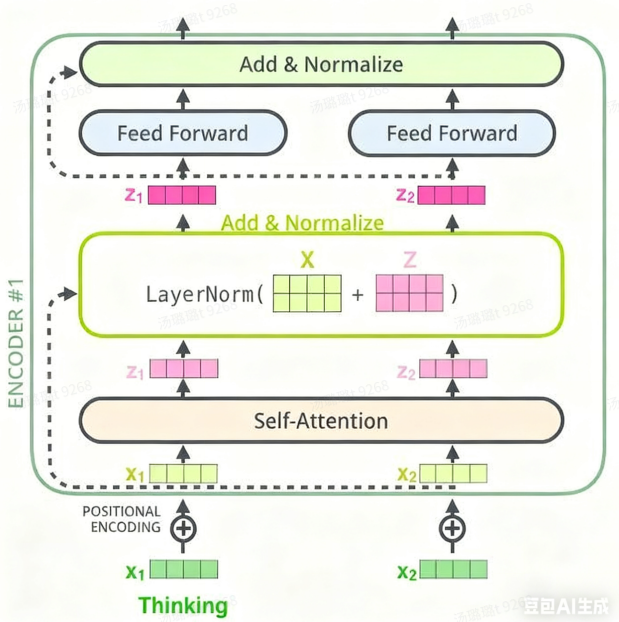
  $$\mathrm{FFN}(x)=\mathrm{ReLU}(xW_1+b_1)W_2+b_2$$

  其中各组件含义：

  - x：输入特征向量（来自注意力层的输出，维度为 d_{model}，即 Transformer 的特征维度）；
  - W_1, b_1：第一层全连接的权重矩阵（维度 $d_{\text{model}}\times d_{\text{ff}}$）与偏置向量（维度 d_{ff}）；
  - W_2, b_2：第二层全连接的权重矩阵（维度 $d_{\text{ff}}\times d_{\text{model}}$）与偏置向量（维度 d_{model}）；
  - $\mathrm{ReLU}(0,\cdot)$：ReLU 激活函数（引入非线性，让模型学习复杂特征）；
  - d_{ff}：FFN 中间层的维度（远大于 d_{model}，目的是通过 “升维 - 非线性变换 - 降维” 增强特征表达能力）。
- 在标准实现中，第一层将维度从 d_{model} 扩展到 d_{ff}（通常为 2048），第二层再恢复为 d_{model}（通常为 512），这种 "扩展 - 收缩" 结构增强了模型的特征表达能力。实际应用中常加入 Dropout 防止过拟合：$\mathrm{FFN}(x)=\mathrm{Dropout}(\mathrm{ReLU}(xW_1)W_2)$

残差连接+层归一化（LN）

Transformer 是典型的深层模型，而深层模型易出现**梯度消失**和**训练不稳定**。残差连接与层归一化的组合，正是为解决这两个核心问题。



1. 残差连接

i. Transformer 中的残差连接位置

- 在 Transformer 的编码器 / 解码器中，每个子层（注意力层、FFN 层）都配有一个残差连接和层标准化（layer-normalization），具体流程：
  - 对于注意力层：设输入为 x，注意力层的输出为 $\mathrm{Attention}(x)$，则残差连接后的结果为 $x + \mathrm{Attention}(x)$；
  - 对于 FFN 层：设输入为 $y = x + \mathrm{Attention}(x)$（注意力层残差后的结果），FFN 的输出为 $\mathrm{FFN}(y)$，则残差连接后的结果为 $y + \mathrm{FFN}(y)$。

ii. 残差连接的核心原理（解决梯度消失）

反向传播时，梯度会沿着 “残差路径” 直接传递到浅层：

- 假设某层的输出为 $o = f(x) + x$（$(f(x))$ 是该层的变换，如注意力层 / FFN 的输出）；
- 对输入 x 求导得：$\dfrac{\partial o}{\partial x} = \dfrac{\partial f(x)}{\partial x} + 1$；
- 即使 $\dfrac{\partial f(x)}{\partial x}$ 经多层传播后趋近于 0，整体梯度仍会保留 “1” 这个基础值，避免梯度消失，让浅层参数能有效更新。

2. 层归一化（LN）

- 核心思想：对每个样本的特征维度进行标准化（均值为 0、方差为 1），消除 “内部协变量偏移”，让每层输入的分布更稳定，从而加速训练。
  - LN(x) = γ * x_norm + β
- LayerNorm vs BatchNorm

维度	BatchNorm	LayerNorm
统计范围	批次内所有样本的 “相同特征位置”	单个样本的 “所有特征位置”
批次大小影响	强依赖：批次变化会导致统计量剧变	无依赖：批次变化不影响统计量
适用场景	批次固定的场景（如 CNN 训练）	批次动态变化的场景（如序列推理）

- layernorm:归一化逻辑是：对单个样本，在 “所有特征位置” 上计算均值和方差，完全不依赖批次内的其他样本。
  - 为什么适合批次动态变化的场景？
    - A：归一化的结果仅由当前样本自身决定，不受批次内其他样本影响，因此批次大小变化时，数据分布稳定.
  - 为什么适合序列 / 动态场景？
    - 特征位置意义不固定：文本中 “第 5 个 token” 在不同样本中可能是完全不同的语义（比如一个是名词，一个是动词），跨样本统计没有意义；
- batchnorm:归一化逻辑是：对同一批次内的所有样本，在 “相同特征位置” 上计算均值和方差。
  - 为什么适合 CNN 训练？
    - 批次大小稳定：训练时通常固定批次（如 32、64），不会频繁变化，因此批次内的统计量（均值 / 方差）可靠。
    - 特征位置意义固定：图像的每个 `(C, H, W)` 位置（如 “红色通道的 (10,20) 像素”）在不同样本中具有相似的语义（都是图像某个区域的颜色信息），跨样本统计是合理的（相当于对同类特征做 “全局校准”）。
- 为什么transformer中使用LayerNorm而不是BatchNorm
  - 答：选择哪种归一化方法取决于具体任务和模型架构。例如，Transformer 选择 LayerNorm 而非 BatchNorm，是因为处理变长序列时 LayerNorm 更稳定；而 CNN 在图像任务中 BatchNorm 仍是主流选择

解码器（Decoder）

- **整体结构：与编码器的异同：**

  1. 新增 “掩码自注意力层” （解决生成顺序依赖）；
  2. 新增 “编码器 - 解码器注意力层” （关联源 - 目标序列）

  这两个模块是解码器实现 “有序生成” 和 “上下文关联” 的核心，也是理解解码器的重点。

- **模块 1：掩码多头自注意力层（Masked Multi-Head Attention）**

- 核心问题：解决 “生成顺序依赖” ：解码器生成目标序列时，**只允许关注到输出序列中早于当前位置之前的单词**（否则违背 “顺序生成” 逻辑）。
- “掩码” 的实现方式：构建一个下三角掩码矩阵（对角线及以下为 1，对角线以上为 -∞ 或 0），通过 “遮挡” 后续位置的注意力分数，让模型无法关注未生成的词。

- **模块 2：编码器 - 解码器注意力层（Encoder-Decoder Attention，简称 “交叉注意力层”）**

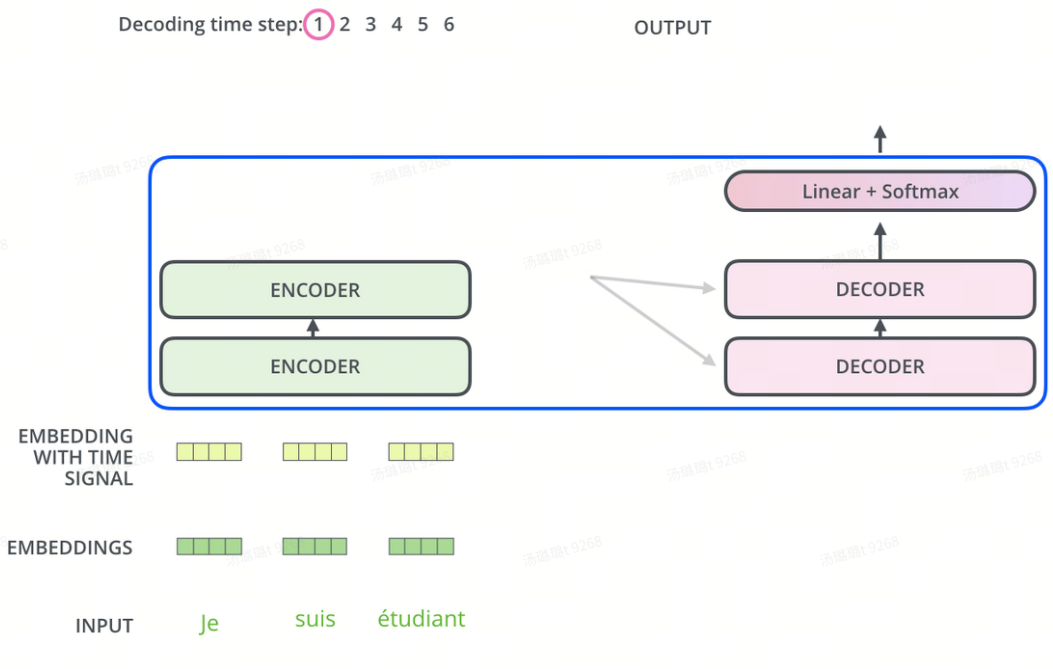
  1. 核心作用：关联 “源序列上下文” 与 “目标序列当前生成内容”

     解码器生成目标序列时，不仅需要依赖已生成的目标词（由掩码自注意力层处理），还需要结合编码器输出的源序列全局上下文（如翻译 “apple” 时，需结合源句中 “红色” “水果” 等上下文信息）。

  2. Decoder Attention层：原理和多头注意力（multiheaded Self Attention）机制类似，不同之处是：Decoder Attention层是使用前一层的输出来构造 Query 矩阵，而 Key 矩阵和 Value 矩阵来自于编码器（Encoder）最终的输出。

- **编码器与解码器的协同合作**

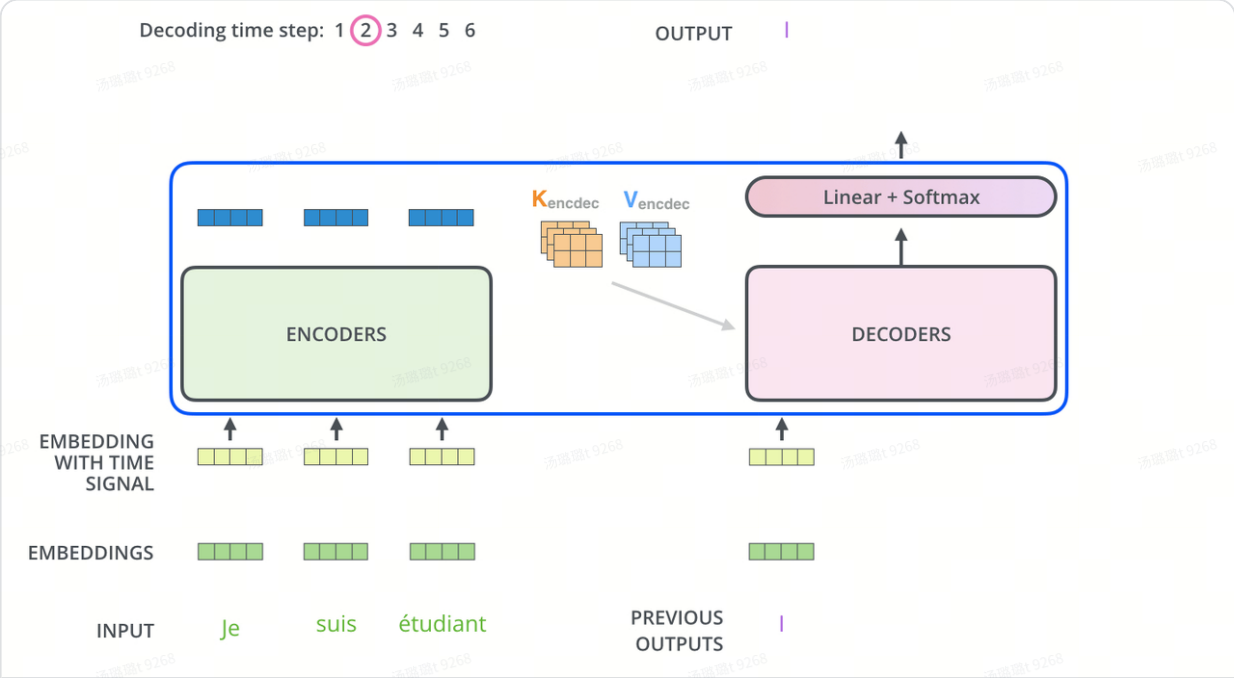
  - **编码器**首先处理输入序列。然后将顶部编码器的输出转换为一组注意力向量 K 和 V。每个解码器在其“编码器-解码器注意力”层中使用这些，以帮助解码器专注于输入序列中的适当位置。



transformer_decoding_1.gif

  - 在完成了编码（encoding）阶段之后，我们开始解码（decoding）阶段。解码（decoding ）阶段的每一个时间步都输出一个翻译后的单词（这里的例子是英语翻译）。

- 接下来会重复这个过程，直到输出一个结束符，Transformer 就完成了所有的输出。每一步的输出都会在下一个时间步输入到下面的第一个解码器。**Decoder** 就像 Encoder 那样，从下往上一层一层地输出结果。正对如编码器的输入所做的处理，我们把解码器的输入向量，也加上位置编码向量，来指示每个词的位置。



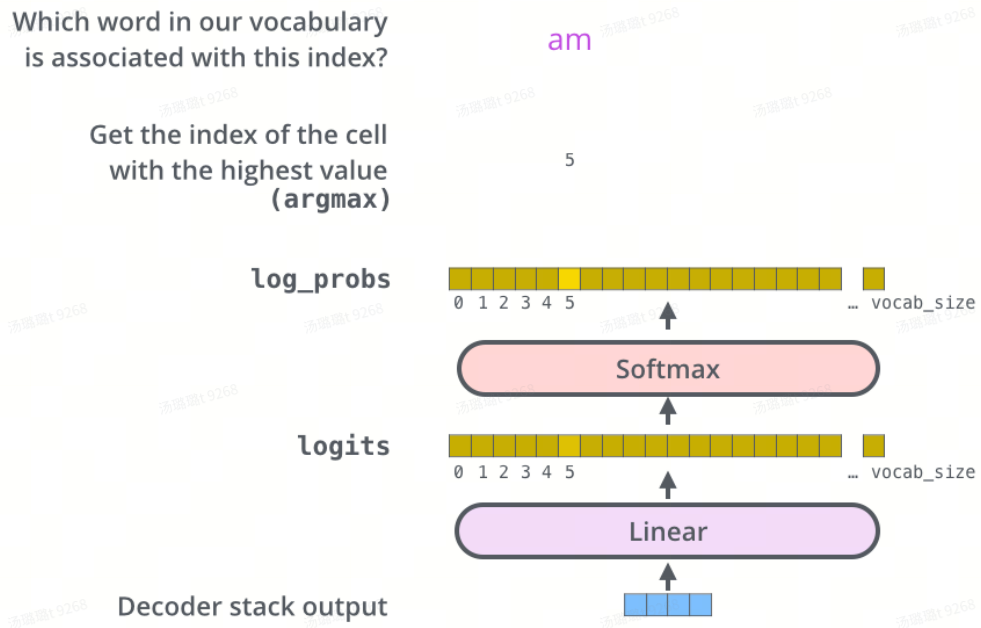
transformer_decoding_2.gif

解码器最终输出层（线性层+Softmax）

- Decoder 最终的输出是一个向量，这组向量仍是 “数值形式”。我们怎么把这个向量转换为单词呢？这是由 线性层+Softmax 层 来完成的。
  - 线性层：负责 “维度映射” —— 把解码器输出的低维特征向量，放大到与 “目标词表大小” 一致的维度；
  - Softmax 层：负责 “概率转换” —— 把线性层输出的 “分数” 转化为 “概率”，方便选择 “最可能的单词”。

> 🍞 • 协同完成转换流程：“数值→分数→概率→单词”

- 线性层：一个普通的全连接神经网络，可以把解码器输出的向量，映射到一个更长的向量（logits 向量）。
  - 现在假设我们的模型有 10000 个英语单词（模型的输出词汇表），这些单词是从训练集中学到的。因此 logits 向量有 10000 个数字，每个数表示一个单词的分数。我们就是这样去理解线性层的输出。
- Softmax 层：会把这些分数转换为概率（把所有的分数转换为正数，并且加起来等于 1）。然后选择最高概率的那个数字对应的词，就是这个时间步的输出单词。



引用：

课程链接：http://jalammar.github.io/illustrated-transformer/

论文：Vaswani, Ashish, et al. "Attention is All You Need." *Neural Information Processing Systems*, 2017.

1